Automated E-Commerce Platform

1. Project Planning & Management

1.1 Idea

The project is a React + Node.js/Express + PostgreSQL online store (products, categories, cart, JWT login, Stripe test checkout). Today it only runs manually on a developer's machine. Our project adds a **basic DevOps workflow** around it: run everything in Docker, build and test automatically on every push, and deploy a working version.

1.2 Objectives

- Put the frontend, backend and database in Docker and run them with one command (Docker Compose).
- Set up a simple CI pipeline (Jenkins): install, test, build on every push / pull request.
- Push the Docker images to a registry (Docker Hub) with version tags.
- Deploy the app on one server or VM using the pipeline or a simple script.
- Keep secrets (DB password, JWT secret, Stripe keys) out of Git.
- Write a short README so anyone can run the project.

1.3 Scope

In scope (version 1)	Out of scope (possible later)
Dockerfiles + Docker Compose Basic CI/CD with Jenkins Docker Hub images Deployment to one VM Environment variables / GitHub Secrets Basic health check and simple logs	Kubernetes, Helm Terraform / Ansible Prometheus and Grafana New shop features or UI redesign Real payments (Stripe test mode only)

1.4 Timeline (6 weeks)

Week	Work	Result
1	Planning, read the code, run it locally	app runs locally, task board
2	Dockerize	Dockerfiles, Docker Compose, `.env.example`
3	CI pipeline	Jenkins: install, test, build on every push
4	Registry + CD	Images pushed to Docker Hub; deployment to a VM
5	Secrets, health check, fixes	Secrets in GitHub Secrets; logs and health endpoint
6	Final testing and delivery	Final docs, demo, presentation

1.5 Team Roles

Member	Role	Responsibilities	Deliverables
أدهم جمال فؤاد	Team Lead & Project Manager	Overall project planning, task tracking, team coordination, risk management, compiling official documentation, and leading the final presentation	Project Plan, Gantt Chart, documentation, and Final Presentation slides
حسن يحيي حسن	Frontend & Containerization Engineer	Dockerizing the React frontend application, optimizing Nginx/web server configuration inside container, and managing frontend environment variables	Frontend Dockerfile, Nginx config, and optimized frontend image
أحمد السيد عاشور	Backend & Database Containerization Engineer	Dockerizing Node.js/Express backend, setting up PostgreSQL container, database volumes, and multi-service orchestration with Docker Compose	Backend Dockerfile, docker-compose.yml, and .env.example setup
عبدالله أنور إبراهيم	CI Pipeline Engineer	Designing and maintaining GitHub Actions CI workflow (automated linting,	Jenkins CI workflow (.github/workflows/ci.yml)

		code installation, running unit/smoke tests on every push & pull request)	
محمد ابراهيم السيد	CD & Deployment Engineer	Automating Docker Hub image build & push with semantic version tagging, and setting up automated deployment to the target Virtual Machine (VM)	Jenkins CD workflow (.github/workflows/cd.yml) and deployment scripts
أحمد مصطفي إسماعيل	QA, Security & Compliance Engineer	Implementing basic API smoke/health tests, managing GitHub Secrets (JWT, Stripe, DB credentials), verifying container security, and writing setup README	Test suite, GitHub Secrets configuration, security review, and verified README.md

Everyone reviews pull requests; nobody merges their own change into `main`.

1.6 Risks

Risk	Handling
Secrets pushed to GitHub by mistake	Use `.gitignore` and GitHub Secrets; rotate any key that leaks
App behaves differently on each machine	Run everything in Docker with pinned versions
No tests exist in the project	Add a few simple API smoke tests
Team members are new to the tools	Pair on the first Dockerfile and pipeline; keep scope small
No budget for a server	Use a free tier or a local VM

1.7 Success Indicators

Indicator	Target
Run the full stack from a clean machine	One command: `docker compose up`
CI result after a push	Within 10 minutes
Deployment	Automatic or one command, no manual file editing
Secrets in the repository	None
Someone outside the team can set it up	Yes, using the README only

2. Literature Review

We reviewed common e-commerce solutions to see what they offer and where our project fits.

Solution	Pros	Cons	Link to our project
Shopify (hosted)	Fully managed, reliable, no server work	Monthly and transaction fees; no infrastructure control	Shows the reliability users expect
WooCommerce (WordPress)	Free, many plugins, popular	Setup and updates often manual; depends on hosting quality	Typical self-hosted shop without automation
Magento / Adobe Commerce	Very powerful and customizable	Heavy and complex to run	Shows how complexity raises operations cost
Medusa.js (Node.js)	Open source, Node-based, API-first	You still design deployment yourself	Closest to our stack
Simple React + Node repos (like ours)	Easy to learn; core features done	Run locally only; no Docker, no pipeline	This is the gap we fill

2.1 What We Learn

- Hosted platforms are reliable because deployment is automated, so we automate build and deployment.
- Self-hosted stores suffer from manual setup, so we keep everything in Docker and in the repository.
- Our difference: same app features, but with a repeatable run, test and deploy process.

2.2 References

Shopify – shopify.com | WooCommerce – woocommerce.com | Medusa.js – docs.medusajs.com | Docker docs – docs.docker.com | GitHub Actions docs – docs.github.com/actions | Stripe docs – docs.stripe.com | Base app – github.com/3bdallah99/react-node-ecommerce-devops | Kim et al., *The DevOps Handbook* (2016).

Platform descriptions come from general public knowledge; check each vendor's site before final submission.

3. Requirements Gathering

3.1 Stakeholders

Stakeholder	Needs
Shoppers	Working site, cart and checkout, fast pages
Store admin	Add and edit products safely; data is not lost
Developers	Easy local setup, quick pipeline feedback
Our DevOps team	Repeatable build and deployment, readable logs
DEPI evaluators	Working demo and clear documentation

3.2 Functional Requirements

- FR1: The existing app features (login, products, categories, cart, Stripe test checkout) keep working.
- FR2: Frontend, backend and PostgreSQL run with one Docker Compose command.
- FR3: Every push or pull request runs install, test and build automatically.
- FR4: Merging to `main` builds Docker images and pushes them to Docker Hub with a version tag.
- FR5: The latest images can be deployed to a server with one command or automatically.
- FR6: Configuration and secrets come from environment variables, not from code.
- FR7: The backend has a health-check endpoint and writes logs to the console.

3.3 Quality Requirements

Quality	Requirement
Performance	Normal API requests respond in under 1 second; pipeline finishes in about 10 minutes
Security	No secrets in Git; passwords hashed; containers do not use default passwords
Usability	README explains setup in a few steps; clear pipeline error messages
Reliability	Failed containers restart automatically; database data kept in a Docker volume
Maintainability	All Docker and pipeline files stored in the repository; simple folder structure

3.4 Constraints

- About 6 weeks, alongside other DEPI work; four team members; free tools only.
- Technology is fixed by the existing app (React, Node.js, PostgreSQL, Stripe test mode).